NAZON

FOREX TRADING ANALYSER

Python Mastery Guide

Beginner → Intermediate → Pro

Packed with Examples You Can Run in VS Code

Pairs	XAUUSD · GBPUSD · EURUSD · USDJPY
Style	Swing Trading — 4H / Daily
Platform	MT5 + Exness Demo → Live
AI Engine	Claude AI by Anthropic
Lessons	20 Full Lessons + Dozens of Examples

Every code block looks like VS Code. Run each one as you read.

TABLE OF CONTENTS

BEGINNER

L01 — Variables & Data Types

L02 — Strings & F-Strings

L03 — Lists

L04 — Dictionaries

L05 — Conditionals (if / elif / else)

L06 — Loops (for & while)

L07 — Functions (Deep Dive)

L08 — Libraries & Imports

INTERMEDIATE

L09 — Classes & Object-Oriented Programming

L10 — Error Handling (try / except)

L11 — Working with Files & CSV

L12 — pandas & DataFrames

L13 — .env Files & Config Management

L14 — APIs & the Claude AI Integration

PRO

L15 — Advanced OOP & Module Design

L16 — Decorators & Closures

L17 — Threading & Scheduling

L18 — SQLite Trade Journal

L19 — MT5 Library Deep Dive

L20 — NAZON v1 — Full Working Scanner

B

BEGINNER

Python Foundations for NAZON

Work through every example. Type them yourself — don't copy-paste. That's how it sticks.

LESSON 01

Variables & Data Types

NAZON uses variables for prices, balances, pair names, lot sizes and signals.

Think of a variable as a labelled box. You put a value in, give it a name, and use that name to get the value back.

Example 1 — Your First Variables

```
# Create variables
name = 'NAZON'
version = 1
balance = 1000.00
running = True

# Print them
print(name)
print(version)
print(balance)
print(running)
```

Output:

```
NAZON
1
1000.0
True
```

Example 2 — NAZON Trading Variables

```
# These are real variables NAZON uses
pair = 'XAUUSD' # the pair we are trading
price = 2345.50 # current Gold price
lot_size = 0.04 # how many lots to trade
stop_loss = 2321.00 # SL price
take_profit = 2394.50 # TP price
connected = False # is MT5 connected?
signal = 'BUY' # trade direction
confidence = 82 # AI confidence score

print(pair)
print(price)
print(signal)
print(confidence)
```

```
XAUUSD
2345.5
BUY
82
```

Example 3 — Updating Variables

```
# Variables can change value
balance = 1000.00
print(balance)

# After a winning trade
balance = balance + 20.00
print(balance)

# Shorthand way
balance += 15.00
print(balance)

# MT5 just connected
connected = False
connected = True
print(connected)
```

```
1000.0
1020.0
1035.0
True
```

Example 4 — The 4 Data Types

```
# str — text (in quotes)
pair = 'XAUUSD'
signal = 'BUY'
name = 'NAZON Forex Analyser'

# float — decimal number
price = 2345.50
lot_size = 0.04
risk_pct = 0.01

# int — whole number
candles = 50
rsi_period = 14
trades_won = 7

# bool — True or False only
connected = True
trade_active = False
signal_ready = True

# Check the type of any variable
print(type(pair))
print(type(price))
print(type(candles))
print(type(connected))
```

```
<class 'str'>
<class 'float'>
<class 'int'>
<class 'bool'>
```

Example 5 — Math with Variables

```
entry = 2345.50
sl = 2321.00
tp = 2394.50

# Calculate pips at risk and reward
risk_pips = entry - sl
reward_pips = tp - entry
rr_ratio = reward_pips / risk_pips

print(f'Risk : {risk_pips} points')
print(f'Reward : {reward_pips} points')
print(f'R:R : 1 : {round(rr_ratio,2)}')
```

```
Risk : 24.5 points
Reward : 49.0 points
R:R : 1 : 2.0
```

⌨■ PRACTICE: Variables

1. Create a variable for each of your 4 pairs: pair1='XAUUSD', pair2=...
2. Create: entry=1.2745, sl=1.2680, tp=1.2880 for GBPUSD.
3. Calculate and print the risk pips, reward pips and R:R ratio.
4. Create a bool called valid_trade = True and print it.
5. Change balance from 1000 to 1050 using +=, print before and after.

LESSON 02

Strings & F-Strings

NAZON uses strings for pair names, AI prompts, log messages and signals.

Example 1 — String Basics

```
# Strings use single or double quotes
pair1 = 'XAUUSD'
pair2 = "GBPUSD"
msg = 'NAZON is online!'

print(pair1)
print(pair2)
print(msg)
```

```
XAUUSD
GBPUSD
NAZON is online!
```

Example 2 — Joining Strings

```
pair = 'XAUUSD'
signal = 'BUY'

# Old way — joining with +
msg = 'Signal for ' + pair + ' is: ' + signal
print(msg)

# Better way — f-strings (use this in NAZON!)
msg = f'Signal for {pair} is: {signal}'
print(msg)
```

```
Signal for XAUUSD is: BUY
Signal for XAUUSD is: BUY
```

Example 3 — F-Strings with Numbers

```
pair = 'XAUUSD'
price = 2345.50
rsi = 68.5
confidence = 82

print(f'Pair : {pair}')
print(f'Price : {price}')
print(f'RSI : {rsi}')
print(f'Confidence: {confidence}%')

# Round to 2 decimal places inside f-string
lot = 0.041667
print(f'Lot size : {lot:.2f}')
```

```
Pair : XAUUSD
Price : 2345.5
RSI : 68.5
Confidence: 82%
Lot size : 0.04
```

Example 4 — Building the AI Prompt (Real NAZON Code)

```
# This is how NAZON sends data to Claude AI
pair = 'XAUUSD'
price = 2345.50
rsi = 68.5
macd = 0.0012
ema200 = 2300.00
balance = 1000.00

prompt = f'''
You are NAZON AI, a professional forex analyst.
Analyse the following setup for swing trading:

Pair : {pair}
Price : {price}
RSI(14) : {rsi}
MACD : {macd}
EMA 200 : {ema200}
Balance : ${balance}

Give a signal: BUY, SELL, or WAIT.
Include confidence (0-100) and one reason.
'''

print(prompt)
```

```
You are NAZON AI, a professional forex analyst.
Analyse the following setup for swing trading:

Pair : XAUUSD
Price : 2345.5
RSI(14) : 68.5
MACD : 0.0012
EMA 200 : 2300.0
Balance : $1000.0

Give a signal: BUY, SELL, or WAIT.
Include confidence (0-100) and one reason.
```

Example 5 — String Methods

```
pair = ' xauusd '

# Remove spaces
print(pair.strip())

# Make uppercase
print(pair.strip().upper())

# Make lowercase
print('GBPUSD'.lower())

# Replace part of a string
print('XAUUSD'.replace('XAU','GBP'))

# Check if something is inside
print('XAU' in 'XAUUSD')
print('JPY' in 'XAUUSD')

# Length of a string
print(len('XAUUSD'))

# Split a string
msg = 'SIGNAL:BUY:82'
parts = msg.split(':')
print(parts)
```

```
xauusd
XAUUSD
gbpusd
GBPUSD
True
False
6
['SIGNAL', 'BUY', '82']
```

⇒■ PRACTICE: Strings

1. Create pair='gbpusd' and print it uppercase using .upper().

2. Use an f-string to print: 'GBPUSD entry: 1.2745 | SL: 1.2680 | TP: 1.2880'.

3. Build a 5-line AI prompt for EURUSD using f-strings with fake values.

4. Use .split(':') on 'PAIR:XAUUSD:SIGNAL:BUY' and print each part.

5. Check if 'USD' is in each of your 4 pairs using the 'in' keyword.

LESSON 03

Lists

NAZON stores pairs, candle prices and indicator values in lists.

Example 1 — Creating and Accessing Lists

```
# NAZON's pairs list
pairs = ['XAUUSD', 'GBPUSD', 'EURUSD', 'USDJPY']

# Access items by index (starts at 0)
print(pairs[0])
print(pairs[1])
print(pairs[2])
print(pairs[3])
print(pairs[-1])
print(pairs[-2])
```

```
XAUUSD
GBPUSD
EURUSD
USDJPY
USDJPY
EURUSD
```

Example 2 — List Operations

```
pairs = ['XAUUSD', 'GBPUSD', 'EURUSD', 'USDJPY']

# How many items?
print(len(pairs))

# Add a new pair
pairs.append('EURJPY')
print(pairs)

# Remove a pair
pairs.remove('EURJPY')
print(pairs)

# Is a pair in the list?
print('XAUUSD' in pairs)
print('BTCUSD' in pairs)

# Slice — get first 2 pairs
print(pairs[:2])
```

```
4
['XAUUSD', 'GBPUSD', 'EURUSD', 'USDJPY', 'EURJPY']
['XAUUSD', 'GBPUSD', 'EURUSD', 'USDJPY']
True
False
['XAUUSD', 'GBPUSD']
```

Example 3 — Candle Closing Prices

```
# Last 5 closing prices for XAUUSD (newest last)
closes = [2330.0, 2338.5, 2345.0, 2352.5, 2345.5]

print('Highest:', max(closes))
print('Lowest :', min(closes))
print('Latest :', closes[-1])

# Average
avg = sum(closes) / len(closes)
print(f'Average: {round(avg,2)}')

# Is price trending up? (last > first)
trending_up = closes[-1] > closes[0]
print(f'Trending up: {trending_up}')
```

```
Highest: 2352.5
Lowest : 2330.0
Latest : 2345.5
Average: 2342.3
Trending up: True
```

Example 4 — List of Signals

```
# Store signals for all pairs
signals = ['BUY', 'SELL', 'WAIT', 'BUY']
pairs = ['XAUUSD','GBPUSD','EURUSD','USDJPY']

# Count how many BUY signals
buy_count = signals.count('BUY')
print(f'BUY signals today: {buy_count}')

# Find index of GBPUSD
idx = pairs.index('GBPUSD')
print(f'GBPUSD signal: {signals[idx]}')

# Sorted list
print(sorted(pairs))
```

```
BUY signals today: 2
GBPUSD signal: SELL
['EURUSD', 'GBPUSD', 'USDJPY', 'XAUUSD']
```

⌨■ PRACTICE: Lists

1. Create a list of your 4 pairs. Print item at index 0 and the last item.

2. Create closes=[2340,2338,2345,2350,2347]. Find max, min, average.

3. Add 'EURJPY' to your pairs list, print it, then remove it.

4. Create a signals list ['BUY','WAIT','SELL','BUY']. Count the BUY signals.

5. Slice your pairs list to get only the first 2 pairs.

LESSON 04

Dictionaries

Every candle, signal, account snapshot and indicator in NAZON is a dictionary.

Example 1 — Your First Dictionary

```
# Dictionary: key → value pairs
candle = {
'open' : 2340.00,
'high' : 2352.50,
'low' : 2337.20,
'close' : 2345.50,
'volume': 18420
}

# Access by key
print(candle['open'])
print(candle['close'])
print(candle['high'])
```

```
2340.0
2345.5
2352.5
```

Example 2 — Adding, Updating, Removing Keys

```
candle = {'open':2340.0, 'high':2352.5, 'low':2337.2, 'close':2345.5}

# Add a new key
candle['bullish'] = candle['close'] > candle['open']
print(candle['bullish'])

# Add body size
candle['body'] = abs(candle['close'] - candle['open'])
print(f"Body: {candle['body']}")

# Update a value
candle['close'] = 2348.0
print(candle['close'])

# All keys
print(candle.keys())

# All values
print(list(candle.values()))
```

```
True
Body: 5.5
2348.0
dict_keys(['open', 'high', 'low', 'close', 'bullish', 'body'])
[2340.0, 2352.5, 2337.2, 2348.0, True, 8.0]
```

Example 3 — The NAZON Signal Dictionary

```
# This is the most important dictionary in NAZON
# signal_engine.py returns this for every pair
signal = {
'pair' : 'XAUUSD',
'timeframe' : 'H4',
'direction' : 'BUY',
'entry' : 2345.50,
'stop_loss' : 2321.00,
'take_profit' : 2394.50,
'rsi' : 68.5,
'macd' : 0.0012,
'confidence' : 82,
'reason' : 'Bullish structure, RSI pullback from oversold'
}

# Use the signal
print(f"{signal['pair']} → {signal['direction']}")
print(f"Entry : {signal['entry']}")
print(f"SL : {signal['stop_loss']}")
print(f"TP : {signal['take_profit']}")
print(f"Conf : {signal['confidence']}%")
print(f"Why : {signal['reason']}")
```

```
XAUUSD → BUY
Entry : 2345.5
SL : 2321.0
TP : 2394.5
Conf : 82%
Why : Bullish structure, RSI pullback from oversold
```

Example 4 — NAZON Account Snapshot Dictionary

```
# What NAZON reads from MT5 every scan
account = {
'balance' : 1000.00,
'equity' : 1024.50,
'profit' : 24.50,
'margin_used' : 10.20,
'free_margin' : 1014.30,
'open_trades' : 1,
'daily_pnl' : 24.50,
}

print(f"Balance : ${account['balance']}")
print(f"Equity : ${account['equity']}")
print(f"Profit : ${account['profit']}")
print(f"Open trades: {account['open_trades']}")

# Calculate account growth
growth = (account['profit'] / account['balance']) * 100
print(f"Growth today: {round(growth,2)}%")
```

```
Balance : $1000.0
Equity : $1024.5
Profit : $24.5
Open trades: 1
Growth today: 2.45%
```

Example 5 — Dictionary of Dictionaries (Nested)

```
# NAZON stores all pair signals in one big dictionary
all_signals = {
'XAUUSD': {'direction':'BUY', 'confidence':82, 'entry':2345.50},
'GBPUSD': {'direction':'SELL', 'confidence':75, 'entry':1.2745},
'EURUSD': {'direction':'WAIT', 'confidence':45, 'entry':1.0850},
'USDJPY': {'direction':'BUY', 'confidence':70, 'entry':149.50},
}

# Access nested values
print(all_signals['XAUUSD']['direction'])
print(all_signals['GBPUSD']['confidence'])

# Loop through all pairs
for pair, data in all_signals.items():
print(f"{pair}: {data['direction']} ({data['confidence']}%)")
```

```
BUY
75
XAUUSD: BUY (82%)
GBPUSD: SELL (75%)
EURUSD: WAIT (45%)
USDJPY: BUY (70%)
```

⌨■ PRACTICE: Dictionaries

1. Create a candle dict for GBPUSD with open, high, low, close, volume.
2. Add key 'bullish' = True if close > open.
3. Create the full signal dict for EURUSD with all keys from Example 3.
4. Print only pair, direction and confidence using f-strings.
5. Create all_signals with all 4 pairs and loop through printing each signal.

LESSON 05

Conditionals (if / elif / else)

NAZON makes decisions: Is RSI oversold? Is RR good enough? Should we trade?

Example 1 — Basic if/elif/else

```
rsi = 28.5

if rsi > 70:
print('RSI is OVERBOUGHT — possible SELL signal')
elif rsi < 30:
print('RSI is OVERSOLD — possible BUY signal')
else:
print('RSI is NEUTRAL — wait for clearer signal')
```

```
RSI is OVERSOLD — possible BUY signal
```

Example 2 — Signal Logic (Real NAZON Code)

```
# core/signal_engine.py logic
rsi = 65.0
macd = 0.0015 # positive = bullish momentum
trend = 'bullish' # price is above EMA 200
price = 2345.50
ema200 = 2300.00

if trend == 'bullish' and rsi < 70 and macd > 0:
signal = 'BUY'
elif trend == 'bearish' and rsi > 30 and macd < 0:
signal = 'SELL'
else:
signal = 'WAIT'

print(f'NAZON Signal: {signal}')
```

```
NAZON Signal: BUY
```

Example 3 — Risk Management Checks

```
# core/risk_manager.py — NAZON runs these before every trade
balance = 1000.00
daily_loss = -45.00 # how much lost today
max_loss = -50.00 # daily loss limit
rr_ratio = 2.0 # risk:reward
confidence = 82 # AI confidence

if daily_loss <= max_loss:
print('BLOCKED: Daily loss limit reached!')
elif rr_ratio < 1.5:
print('BLOCKED: Risk:Reward too low!')
elif confidence < 65:
print('BLOCKED: AI not confident enough!')
else:
print('APPROVED: Trade meets all NAZON criteria!')
```

```
APPROVED: Trade meets all NAZON criteria!
```

Example 4 — Combining Conditions

```
# 'and' — BOTH must be True
rsi = 65
trend = 'bullish'

if rsi < 70 and trend == 'bullish':
print('Both conditions met — BUY possible')

# 'or' — at least ONE must be True
pair = 'XAUUSD'

if pair == 'XAUUSD' or pair == 'GBPUSD':
print(f'{pair} is a NAZON priority pair')

# 'not' — reverses True/False
connected = False

if not connected:
print('MT5 is not connected — reconnecting...')
```

```
Both conditions met — BUY possible
XAUUSD is a NAZON priority pair
MT5 is not connected — reconnecting...
```

Example 5 — Nested if Statements

```
pair = 'XAUUSD'
confidence = 82
rr_ratio = 2.0
signal = 'BUY'

if signal != 'WAIT':
if confidence >= 70:
if rr_ratio >= 1.5:
print(f'NAZON: Execute {signal} on {pair}')
print(f'Confidence: {confidence}% | RR: 1:{rr_ratio}')
else:
print('RR too low — skip')
else:
print('Confidence too low — skip')
else:
print('Signal is WAIT — do nothing')
```

```
NAZON: Execute BUY on XAUUSD
Confidence: 82% | RR: 1:2.0
```

═■ PRACTICE: Conditionals

1. Write if/elif/else: print 'Overbought' if RSI>70, 'Oversold' if RSI<30, else 'Neutral'.
2. Test 3 different RSI values (28, 50, 75) — each should give a different output.
3. Write a condition: only approve trade if confidence>=70 AND rr_ratio>=1.5.
4. Write nested ifs: check signal != 'WAIT', then confidence, then rr_ratio.
5. Use 'and', 'or', 'not' — each at least once in different examples.

LESSON 06

Loops (for & while)

NAZON scans all 4 pairs automatically. Loops make that happen.

Example 1 — Basic for Loop

```
pairs = ['XAUUSD', 'GBPUSD', 'EURUSD', 'USDJPY']

for pair in pairs:
print(f'Scanning {pair}...')
```

```
Scanning XAUUSD...
Scanning GBPUSD...
Scanning EURUSD...
Scanning USDJPY...
```

Example 2 — Loop with Index (enumerate)

```
pairs = ['XAUUSD', 'GBPUSD', 'EURUSD', 'USDJPY']

for i, pair in enumerate(pairs, 1):
print(f'{i}. Analysing {pair}')
```

```
1. Analysing XAUUSD
2. Analysing GBPUSD
3. Analysing EURUSD
4. Analysing USDJPY
```

Example 3 — Loop with Conditionals (Real NAZON Scan)

```
pairs = ['XAUUSD', 'GBPUSD', 'EURUSD', 'USDJPY']
signals = {}

# Fake RSI values for each pair
rsi_data = {
'XAUUSD': 65.0,
'GBPUSD': 72.5,
'EURUSD': 44.0,
'USDJPY': 28.5,
}

for pair in pairs:
rsi = rsi_data[pair]

if rsi > 70:
signals[pair] = 'SELL'
elif rsi < 30:
signals[pair] = 'BUY'
else:
signals[pair] = 'WAIT'

print(f'{pair}: RSI={rsi} → {signals[pair]}')
```

```
XAUUSD: RSI=65.0 → WAIT
GBPUSD: RSI=72.5 → SELL
EURUSD: RSI=44.0 → WAIT
USDJPY: RSI=28.5 → BUY
```

Example 4 — range() Loop

```
# range(start, stop, step)
for i in range(5):
print(f'Candle {i}')

print('---')

# Count down from 3
for i in range(3, 0, -1):
print(f'Starting in {i}...')
print('NAZON scan started!')
```

```
Candle 0
Candle 1
Candle 2
Candle 3
Candle 4
---
Starting in 3...
Starting in 2...
Starting in 1...
NAZON scan started!
```

Example 5 — while Loop (NAZON Main Loop)

```
import time

scan_count = 0
max_scans = 3 # in real NAZON this runs forever

while scan_count < max_scans:
scan_count += 1
print(f'Scan #{scan_count} — Checking all pairs...')
print(' XAUUSD: BUY')
print(' GBPUSD: WAIT')
print(f' Scan complete. Next scan in 4 hours.')
print()

print('Max scans reached — NAZON stopped.')
```

```
Scan #1 — Checking all pairs...
XAUUSD: BUY
GBPUSD: WAIT
Scan complete. Next scan in 4 hours.

Scan #2 — Checking all pairs...
XAUUSD: BUY
GBPUSD: WAIT
Scan complete. Next scan in 4 hours.

Scan #3 — Checking all pairs...
XAUUSD: BUY
GBPUSD: WAIT
Scan complete. Next scan in 4 hours.

Max scans reached — NAZON stopped.
```

Example 6 — break and continue

```
pairs = ['XAUUSD', 'GBPUSD', 'EURUSD', 'USDJPY']
bad_pair = 'EURUSD' # pretend connection failed for this one

for pair in pairs:
if pair == bad_pair:
print(f'Skipping {pair} — no data')
continue # skip to next pair
print(f'Analysed {pair} successfully')

print('---')

# Stop scanning if a critical error happens
for pair in pairs:
if pair == 'GBPUSD':
print('Critical error — stopping all scans!')
break # exit the loop completely
print(f'Scanned {pair}')
```

```
Analysed XAUUSD successfully
Analysed GBPUSD successfully
Skipping EURUSD — no data
Analysed USDJPY successfully
---
Scanned XAUUSD
Critical error — stopping all scans!
```

⌨■ PRACTICE: Loops

1. Loop through 4 pairs and print 'Checking [PAIR]...' for each.

2. Create fake RSI dict and loop to print BUY/SELL/WAIT for each pair.

3. Use enumerate to number the pairs: '1. XAUUSD, 2. GBPUSD...'

4. Use a while loop that runs 3 times, printing 'Scan #X complete'.

5. Loop through pairs but skip EURUSD using continue.

LESSON 07

Functions (Deep Dive)

Every NAZON module is a collection of functions. Master this lesson well.

Example 1 — Basic Function

```
def greet():
print('NAZON Forex Analyser is online!')
print('Connecting to MT5...')

greet()
```

```
NAZON Forex Analyser is online!
Connecting to MT5...
```

Example 2 — Function with Parameters

```
def analyse_pair(pair, timeframe):
print(f'Analysing {pair} on {timeframe} chart...')
print(f'Fetching last 50 candles for {pair}')

analyse_pair('XAUUSD', 'H4')
analyse_pair('GBPUSD', 'D1')
analyse_pair('EURUSD', 'H4')
```

```
Analysing XAUUSD on H4 chart...
Fetching last 50 candles for XAUUSD
Analysing GBPUSD on D1 chart...
Fetching last 50 candles for GBPUSD
Analysing EURUSD on H4 chart...
Fetching last 50 candles for EURUSD
```

Example 3 — Return Values

```
def calculate_rr(entry, sl, tp):
risk = abs(entry - sl)
reward = abs(tp - entry)
rr = reward / risk
return round(rr, 2)

def get_signal(rsi):
if rsi > 70:
return 'SELL'
elif rsi < 30:
return 'BUY'
else:
return 'WAIT'

# Use the functions
rr = calculate_rr(entry=2345.5, sl=2321.0, tp=2394.5)
print(f'R:R ratio: 1:{rr}')

sig = get_signal(rsi=28.5)
print(f'RSI Signal: {sig}')
```

```
R:R ratio: 1:2.0
RSI Signal: BUY
```

Example 4 — The Lot Size Calculator (Real NAZON)

```
# core/risk_manager.py
def calculate_lot_size(balance, risk_pct, sl_points, pip_value=0.1):
'''
Calculate lot size to risk a % of balance.
balance : account balance in USD
risk_pct : e.g. 0.01 for 1%
sl_points : stop loss distance in points
pip_value : value of 1 pip for this pair
Returns : lot size rounded to 2 decimal places
'''
risk_dollars = balance * risk_pct
lot_size = risk_dollars / (sl_points * pip_value)
return round(lot_size, 2)


# Test it:
balance = 1000.00
sl_pts = 245 # 24.5 points on Gold

lot = calculate_lot_size(balance, 0.01, sl_pts)
print(f'1% risk lot size : {lot}')

lot2 = calculate_lot_size(balance, 0.02, sl_pts)
print(f'2% risk lot size : {lot2}')
```

```
1% risk lot size : 0.04
2% risk lot size : 0.08
```

Example 5 — Default Parameters

```
def scan_pair(pair, timeframe='H4', candles=50, risk=0.01):
print(f'Scanning {pair}')
print(f' Timeframe : {timeframe}')
print(f' Candles : {candles}')
print(f' Risk : {risk*100}%')

# Uses all defaults
scan_pair('XAUUSD')
print()

# Override timeframe only
scan_pair('GBPUSD', timeframe='D1')
print()

# Override multiple
scan_pair('EURUSD', timeframe='D1', candles=100, risk=0.02)
```

```
Scanning XAUUSD
Timeframe : H4
Candles : 50
Risk : 1.0%

Scanning GBPUSD
Timeframe : D1
Candles : 50
Risk : 1.0%

Scanning EURUSD
Timeframe : D1
Candles : 100
Risk : 2.0%
```

Example 6 — Returning Multiple Values

```
def get_trade_levels(entry, risk_pct=0.01, rr=2.0):
'''Returns SL and TP based on entry and RR.'''
risk_points = entry * risk_pct
stop_loss = round(entry - risk_points, 2)
take_profit = round(entry + risk_points * rr, 2)
return stop_loss, take_profit

entry = 2345.50
sl, tp = get_trade_levels(entry)

print(f'Entry : {entry}')
print(f'SL : {sl}')
print(f'TP : {tp}')
```

```
Entry : 2345.5
SL : 2322.05
TP : 2392.41
```

⌨■ PRACTICE: Functions

1. Write get_signal(rsi) returning BUY/SELL/WAIT based on RSI value.

2. Write calculate_rr(entry,sl,tp) returning the R:R as a float.

3. Write is_valid_pair(pair) returning True if pair is in your 4 pairs.

4. Write calculate_lot_size(balance, risk_pct, sl_points) — test with $1000.

5. Write a function returning 3 values: signal, confidence, reason.

LESSON 08

Libraries & Imports

NAZON is powered by external libraries. Learn how to use them.

Example 1 — Three Ways to Import

```
# Way 1: import whole library with alias
import MetaTrader5 as mt5
import pandas as pd
import numpy as np

# Way 2: import specific function only
from datetime import datetime
from os import getenv

# Way 3: import everything (use carefully)
from math import *

# Using them:
now = datetime.now()
print(f'NAZON started at: {now}')
```

Example 2 — datetime Library (Used in Trade Journal)

```
from datetime import datetime

now = datetime.now()
print(now)
print(now.strftime('%Y-%m-%d %H:%M'))
print(now.strftime('%d/%m/%Y'))
print(now.hour)
print(now.minute)
```

```
2026-03-09 01:15:32.441234
2026-03-09 01:15
09/03/2026
1
15
```

Example 3 — os Library (Reading .env)

```
import os

# Set an environment variable (normally done in .env file)
os.environ['NAZON_VERSION'] = '1.0'

# Read it back
version = os.getenv('NAZON_VERSION')
print(f'NAZON Version: {version}')

# Check if a path/file exists
print(os.path.exists('data/trades.db'))
```

Example 4 — Installing & Using pandas

```
# First install in your VS Code terminal:
# pip install pandas

import pandas as pd

# Create a simple table (like a spreadsheet)
data = {
'pair' : ['XAUUSD','GBPUSD','EURUSD','USDJPY'],
'signal' : ['BUY', 'SELL', 'WAIT', 'BUY'],
'confidence': [82, 75, 45, 70],
}
df = pd.DataFrame(data)
print(df)
```

```
pair signal confidence
0 XAUUSD BUY 82
1 GBPUSD SELL 75
2 EURUSD WAIT 45
3 USDJPY BUY 70
```

■ **NOTE:** Installing libraries: open the VS Code terminal (Ctrl+`) and run: pip install MetaTrader5 pandas pandas-ta numpy anthropic streamlit plotly python-dotenv schedule

═■ PRACTICE: Libraries

1. Open VS Code terminal and install: pandas, numpy, python-dotenv.

2. Import datetime and print today's date formatted as DD/MM/YYYY.

3. Import pandas and create a DataFrame with your 4 pairs and fake signals.

4. Use os.getenv() to try reading an env variable — print it.

5. BONUS: Try importing MetaTrader5 — if it works, print 'MT5 ready!'

I

INTERMEDIATE

Building NAZON's Core Modules

You know enough Python to read real code. Now we build the actual NAZON modules.

LESSON 09

Classes & Object-Oriented Programming

Each NAZON module is a Class: MT5Connector, RiskManager, AIAnalyst.

Example 1 — Your First Class

```
class TradeSignal:
def __init__(self, pair, direction, confidence):
self.pair = pair
self.direction = direction
self.confidence = confidence

def is_strong(self):
return self.confidence >= 70

def display(self):
status = 'STRONG' if self.is_strong() else 'WEAK'
print(f'{self.pair}: {self.direction} | {self.confidence}% | {status}')

# Create objects from the blueprint
s1 = TradeSignal('XAUUSD', 'BUY', 82)
s2 = TradeSignal('GBPUSD', 'SELL', 55)
s3 = TradeSignal('EURUSD', 'WAIT', 40)

s1.display()
s2.display()
s3.display()
```

```
XAUUSD: BUY | 82% | STRONG
GBPUSD: SELL | 55% | WEAK
EURUSD: WAIT | 40% | WEAK
```

Example 2 — The RiskManager Class

```
# core/risk_manager.py
class RiskManager:
def __init__(self, balance, risk_pct=0.01, min_rr=1.5):
self.balance = balance
self.risk_pct = risk_pct
self.min_rr = min_rr

def get_risk_amount(self):
return self.balance * self.risk_pct

def calculate_lot(self, sl_points, pip_value=0.1):
risk = self.get_risk_amount()
return round(risk / (sl_points * pip_value), 2)

def is_valid_rr(self, sl, tp, entry):
risk = abs(entry - sl)
reward = abs(tp - entry)
rr = reward / risk
return rr >= self.min_rr

def check_trade(self, sl, tp, entry):
if not self.is_valid_rr(sl, tp, entry):
return False, 'RR too low'
return True, 'Trade approved'


# Create and use it
rm = RiskManager(balance=1000, risk_pct=0.01)

print(f'Risk amount : ${rm.get_risk_amount()}')
print(f'Lot size : {rm.calculate_lot(245)}')

ok, msg = rm.check_trade(entry=2345.5, sl=2321.0, tp=2394.5)
print(f'Trade valid : {ok} — {msg}')
```

```
Risk amount : $10.0
Lot size : 0.04
Trade valid : True — Trade approved
```

Example 3 — The MT5Connector Class (Simplified)

```
# core/mt5_connector.py (simplified for practice — no real MT5 needed)
class MT5Connector:
def __init__(self):
self.connected = False
self.account = None

def connect(self):
# In real code: mt5.initialize()
self.connected = True
self.account = {'balance':1000.0, 'equity':1024.5}
print('Connected to MT5!')
return True

def get_balance(self):
if not self.connected:
return None
return self.account['balance']

def disconnect(self):
self.connected = False
print('Disconnected from MT5.')

def status(self):
state = 'ONLINE' if self.connected else 'OFFLINE'
print(f'MT5 Status: {state}')


conn = MT5Connector()
conn.status()
conn.connect()
conn.status()
print(f'Balance: ${conn.get_balance()}')
conn.disconnect()
conn.status()
```

```
MT5 Status: OFFLINE
Connected to MT5!
MT5 Status: ONLINE
Balance: $1000.0
Disconnected from MT5.
MT5 Status: OFFLINE
```

⌨■ PRACTICE: Classes

1. Create class TradeSignal with pair, direction, confidence. Add display() method.

2. Create class RiskManager with balance, risk_pct. Add get_risk_amount() and calculate_lot().

3. Create class MT5Connector with connect(), disconnect(), status() methods.

4. Instantiate all 3 classes and call every method.

5. BONUS: Add a method to RiskManager that blocks trades if balance < 100.

LESSON 10

Error Handling (try / except)

NAZON never crashes — it handles every MT5 disconnect and API failure gracefully.

Example 1 — Basic try/except

```
# Without error handling — crashes everything
# print(1/0) # ZeroDivisionError: division by zero

# With error handling
try:
result = 1 / 0
except ZeroDivisionError:
print('Cannot divide by zero!')
result = 0

print(f'Result: {result}')
print('App is still running!')
```

```
Cannot divide by zero!
Result: 0
App is still running!
```

Example 2 — Multiple except Blocks

```
def safe_divide(a, b):
try:
result = a / b
return result
except ZeroDivisionError:
print('Error: Cannot divide by zero')
return None
except TypeError:
print('Error: Both inputs must be numbers')
return None

print(safe_divide(10, 2))
print(safe_divide(10, 0))
print(safe_divide(10, 'x'))
```

```
5.0
Error: Cannot divide by zero
None
Error: Both inputs must be numbers
None
```

Example 3 — NAZON MT5 Safe Connection

```
class MT5Connector:
def __init__(self):
self.connected = False

def connect(self):
try:
# Simulate connection attempt
# In real code: mt5.initialize()
print('Attempting MT5 connection...')
self.connected = True
print('Connected successfully!')
return True
except Exception as e:
print(f'Connection failed: {e}')
self.connected = False
return False

def get_candles(self, pair):
try:
if not self.connected:
raise ConnectionError('MT5 not connected')
print(f'Fetching {pair} candles...')
return [] # would return real candles
except ConnectionError as e:
print(f'Cannot fetch {pair}: {e}')
return None

conn = MT5Connector()
conn.get_candles('XAUUSD') # not connected yet
conn.connect()
conn.get_candles('XAUUSD') # now connected
```

```
Cannot fetch XAUUSD: MT5 not connected
Attempting MT5 connection...
Connected successfully!
Fetching XAUUSD candles...
```

Example 4 — try / except / else / finally

```
def scan_pair(pair):
try:
print(f'Scanning {pair}...')
if pair not in ['XAUUSD','GBPUSD','EURUSD','USDJPY']:
raise ValueError(f'{pair} is not a valid NAZON pair')
result = 'BUY'

except ValueError as e:
print(f'Invalid pair error: {e}')
result = None

except Exception as e:
print(f'Unexpected error: {e}')
result = None

else:
# Only runs if NO exception happened
print(f' Signal: {result}')

finally:
# ALWAYS runs — error or not
print(f' Scan of {pair} complete.')

return result

scan_pair('XAUUSD')
print()
scan_pair('BTCUSD')
```

```
Scanning XAUUSD...
Signal: BUY
Scan of XAUUSD complete.

Scanning BTCUSD...
Invalid pair error: BTCUSD is not a valid NAZON pair
Scan of BTCUSD complete.
```

⌨■ PRACTICE: Error Handling

1. Wrap a division by zero in try/except and print a friendly message.

2. Write safe_scan(pair) that raises ValueError if pair not in your pairs list.

3. Add else block (runs on success) and finally block (always runs).

4. Write a function that tries to open a missing file — catch FileNotFoundError.

5. Chain two except blocks: one for ValueError, one for any Exception.

LESSON 11

Working with Files & CSV

NAZON logs every trade to a file. You need to read and write files confidently.

Example 1 — Write and Read a Text File

```
# Write to a file
with open('nazon_log.txt', 'w') as f:
f.write('NAZON Trade Log\n')
f.write('=================\n')
f.write('XAUUSD BUY 2345.50\n')
f.write('GBPUSD SELL 1.2745\n')

# Read it back
with open('nazon_log.txt', 'r') as f:
content = f.read()
print(content)
```

```
NAZON Trade Log
=================
XAUUSD BUY 2345.50
GBPUSD SELL 1.2745
```

Example 2 — Read Line by Line

```
with open('nazon_log.txt', 'r') as f:
lines = f.readlines()

for i, line in enumerate(lines, 1):
print(f'Line {i}: {line.strip()}')
```

```
Line 1: NAZON Trade Log
Line 2: =================
Line 3: XAUUSD BUY 2345.50
Line 4: GBPUSD SELL 1.2745
```

Example 3 — CSV Trade Journal

```
import csv
from datetime import datetime

def log_trade(pair, direction, entry, sl, tp, lot):
with open('trades.csv', 'a', newline='') as f:
writer = csv.writer(f)
writer.writerow([
datetime.now().strftime('%Y-%m-%d %H:%M'),
pair, direction,
entry, sl, tp, lot
])
print(f'Logged: {pair} {direction} @ {entry}')


# Log some trades
log_trade('XAUUSD','BUY', 2345.50,2321.00,2394.50,0.04)
log_trade('GBPUSD','SELL',1.2745, 1.2810, 1.2680, 0.10)
log_trade('USDJPY','BUY', 149.50, 148.20, 151.90, 0.05)

# Read trades back
with open('trades.csv','r') as f:
reader = csv.reader(f)
for row in reader:
print(row)
```

```
Logged: XAUUSD BUY @ 2345.5
Logged: GBPUSD SELL @ 1.2745
Logged: USDJPY BUY @ 149.5
['2026-03-09 01:15', 'XAUUSD', 'BUY', '2345.5', '2321.0', '2394.5',
'0.04']
['2026-03-09 01:15', 'GBPUSD', 'SELL', '1.2745', '1.281', '1.268', '0.1']
['2026-03-09 01:15', 'USDJPY', 'BUY', '149.5', '148.2', '151.9', '0.05']
```

═■ PRACTICE: Files

1. Create nazon_signals.txt and write one signal per line for all 4 pairs.

2. Read the file back and print each line with its line number.

3. Write log_trade() function that appends to trades.csv using csv.writer.

4. Log 3 trades then read them all back and print each row.

5. BONUS: Read trades.csv and count how many BUY vs SELL trades.

LESSON 12

pandas & DataFrames

MT5 candle data comes as a pandas DataFrame. Learn to slice and analyse it.

Example 1 — Create a DataFrame

```
import pandas as pd

# Simulate what MT5 gives us
data = {
'time' :
['2026-01-01','2026-01-02','2026-01-03','2026-01-04','2026-01-05'],
'open' : [2330.0,2338.5,2345.0,2352.0,2348.0],
'high' : [2345.0,2352.0,2360.0,2365.0,2355.0],
'low' : [2325.0,2333.0,2340.0,2345.0,2340.0],
'close' : [2338.5,2345.0,2352.0,2348.0,2355.0],
'volume': [18420, 21300, 19800, 22100, 17500],
}
df = pd.DataFrame(data)
print(df)
```

```
time open high low close volume
0 2026-01-01 2330.0 2345.0 2325.0 2338.5 18420
1 2026-01-02 2338.5 2352.0 2333.0 2345.0 21300
2 2026-01-03 2345.0 2360.0 2340.0 2352.0 19800
3 2026-01-04 2352.0 2365.0 2345.0 2348.0 22100
4 2026-01-05 2348.0 2355.0 2340.0 2355.0 17500
```

Example 2 — Reading Data from DataFrame

```
# Get a single column
closes = df['close']
print('Close prices:')
print(closes.tolist())

# Most recent close
latest = df['close'].iloc[-1]
print(f'Latest close: {latest}')

# Last 3 rows
print(df.tail(3))

# Basic stats
print(f'Highest close: {df["close"].max()}')
print(f'Lowest close: {df["close"].min()}')
print(f'Average close: {df["close"].mean():.2f}')
```

```
Close prices:
[2338.5, 2345.0, 2352.0, 2348.0, 2355.0]
Latest close: 2355.0
... (last 3 rows) ...
Highest close: 2355.0
Lowest close: 2338.5
Average close: 2347.70
```

Example 3 — Adding Indicator Columns

```
# Add useful columns
df['bullish'] = df['close'] > df['open']
df['body_size'] = abs(df['close'] - df['open'])
df['wick_up'] = df['high'] - df[['open','close']].max(axis=1)
df['wick_down'] = df[['open','close']].min(axis=1) - df['low']

print(df[['time','close','bullish','body_size']].to_string())
```

```
time close bullish body_size
0 2026-01-01 2338.5 True 8.5
1 2026-01-02 2345.0 True 6.5
2 2026-01-03 2352.0 True 7.0
3 2026-01-04 2348.0 False 4.0
4 2026-01-05 2355.0 True 7.0
```

Example 4 — pandas-ta Indicators (Real NAZON)

```
# pip install pandas-ta
import pandas_ta as ta

# Add RSI
df['rsi'] = ta.rsi(df['close'], length=3) # length=3 for short data

# Add EMA
df['ema3'] = ta.ema(df['close'], length=3)

# Print results
print(df[['time','close','rsi','ema3']].tail(3).to_string())

# Get latest values
latest_rsi = df['rsi'].iloc[-1]
latest_ema = df['ema3'].iloc[-1]
print(f'Latest RSI: {round(latest_rsi,2)}')
print(f'Latest EMA: {round(latest_ema,2)}')
```

■ **NAZON CONNECTION:** In real NAZON, df comes directly from mt5.copy_rates_from_pos() with 100+ candles. pandas-ta then calculates RSI(14), MACD(12,26,9), EMA(200), BB, ATR — all in one line each.

⇒■ PRACTICE: pandas

1. Create a DataFrame with 5 rows of OHLCV data for XAUUSD manually.

2. Print the close column, the last 2 rows, and the highest close.

3. Add a 'bullish' column and a 'body_size' column.

4. Install pandas-ta and add RSI to your DataFrame.

5. Print the latest RSI value using .iloc[-1].

LESSON 13

.env Files & Config Management

NAZON keeps API keys and credentials safe using .env files — never hardcode secrets.

Step 1 — Create the .env File

```
# Create a file called: .env
# Put it in your project root folder (same level as main.py)
# This is NOT Python — just plain key=value lines:

CLAUDE_API_KEY=sk-ant-your-real-key-here
MT5_LOGIN=123456789
MT5_PASSWORD=YourExnessPassword
MT5_SERVER=Exness-MT5Demo
DEFAULT_RISK=0.01
MAX_DAILY_LOSS=0.05
```

Step 2 — Create config.py to Read the .env

```
# config.py — central settings file for all of NAZON
import os
from dotenv import load_dotenv

# Load the .env file
load_dotenv()

# Credentials (secret — from .env)
CLAUDE_API_KEY = os.getenv('CLAUDE_API_KEY')
MT5_LOGIN = int(os.getenv('MT5_LOGIN', '0'))
MT5_PASSWORD = os.getenv('MT5_PASSWORD')
MT5_SERVER = os.getenv('MT5_SERVER')

# Risk settings (from .env)
DEFAULT_RISK = float(os.getenv('DEFAULT_RISK', '0.01'))
MAX_DAILY_LOSS = float(os.getenv('MAX_DAILY_LOSS', '0.05'))

# Trading config (not secret — hardcoded here is fine)
PAIRS = ['XAUUSD','GBPUSD','EURUSD','USDJPY']
TIMEFRAME = 'H4'
RSI_PERIOD = 14
MIN_RR = 1.5
MIN_CONF = 70
CANDLES = 100
```

Step 3 — Use config.py in Other Modules

```
# In any NAZON file, just import what you need:
from config import PAIRS, TIMEFRAME, MIN_RR, MIN_CONF

# Now use them
for pair in PAIRS:
print(f'Will scan {pair} on {TIMEFRAME}')

print(f'Min RR : {MIN_RR}')
print(f'Min Conf : {MIN_CONF}%')
```

```
Will scan XAUUSD on H4
Will scan GBPUSD on H4
Will scan EURUSD on H4
Will scan USDJPY on H4
Min RR : 1.5
Min Conf : 70%
```

Step 4 — Create .gitignore

```
# Create a file called: .gitignore
# This tells Git NOT to upload these files

.env
__pycache__/
*.pyc
data/trades.db
*.log
```

■■ **WATCH OUT:** NEVER upload your .env file to GitHub. Anyone who gets your Claude API key can spend your Anthropic credits. Anyone with your MT5 password can access your trading account.

═■ PRACTICE: .env & Config

1. Create a .env file with a fake API key and MT5 credentials.
2. Create config.py that loads and prints them (mask the key as bool).
3. Add PAIRS, TIMEFRAME, MIN_RR to config.py.
4. Create another file that imports from config and loops through PAIRS.
5. Create .gitignore and add .env to it.

LESSON 14

APIs & the Claude AI Integration

NAZON sends market data to Claude AI and gets back trade signals.

Example 1 — What Happens When NAZON Calls Claude

```
# This is the flow:
# 1. NAZON collects: pair, price, RSI, MACD, EMA, account balance
# 2. NAZON builds a text prompt describing the market
# 3. NAZON sends it to Claude API
# 4. Claude replies with: SIGNAL, CONFIDENCE, REASON
# 5. NAZON reads the reply and acts on it
```

Example 2 — Simple Claude API Call

```
# pip install anthropic
# Get your API key: console.anthropic.com

import anthropic
import os
from dotenv import load_dotenv

load_dotenv()

client = anthropic.Anthropic(api_key=os.getenv('CLAUDE_API_KEY'))

response = client.messages.create(
model = 'claude-sonnet-4-20250514',
max_tokens = 100,
messages = [
{'role':'user', 'content': 'Say hello from NAZON!'}
]
)

print(response.content[0].text)
```

```
Hello from NAZON! Ready to analyse the forex markets for you.
```

Example 3a — The AIAnalyst Class

```
import anthropic
from config import CLAUDE_API_KEY

class AIAnalyst:
def __init__(self):
self.client = anthropic.Anthropic(api_key=CLAUDE_API_KEY)

def analyse(self, pair, indicators, balance):
prompt = f''' # see Example 3b
response = self.client.messages.create(
model='claude-sonnet-4-20250514',
max_tokens=150,
messages=[{'role':'user','content':prompt}]
)
return response.content[0].text.strip()
```

Example 3b — The AI Prompt Template

```
pair = 'XAUUSD'
price = 2345.50
rsi = 65.0
macd = 0.0012
ema200 = 2300.0
trend = 'bullish'
balance = 1000.00

prompt = f'''
You are NAZON, a professional forex swing trading AI.
Analyse this setup and give a trading signal.

PAIR : {pair} PRICE : {price}
RSI : {rsi} MACD : {macd}
EMA200 : {ema200} TREND : {trend}
BALANCE: ${balance}

Reply: SIGNAL: BUY/SELL/WAIT
CONFIDENCE: 0-100
REASON: one sentence
'''

print(prompt)
```

Example 3c — Using the Analyst

```
analyst = AIAnalyst()

result = analyst.analyse(
pair = 'XAUUSD',
indicators = {
'price' :2345.50,
'rsi' :65.0,
'macd' :0.0012,
'ema200':2300.0,
'trend' :'bullish',
},
balance = 1000.00
)
print(result)
```

```
SIGNAL: BUY
CONFIDENCE: 78
REASON: Price above EMA200 confirms bullish trend with RSI at healthy
level.
```

═■ PRACTICE: APIs

1. Get your free Claude API key from console.anthropic.com.
2. Add CLAUDE_API_KEY to your .env file.
3. Write a script that sends 'Analyse XAUUSD: RSI=65, bullish trend' to Claude.
4. Print Claude's full response.
5. BONUS: Build the full AIAnalyst class and test it with all 4 pairs.

P

PRO

Building NAZON End-to-End

This is where everything comes together. Real production-grade NAZON code.

LESSON 15

Advanced OOP & Module Design

How all NAZON modules connect and communicate with each other.

Example 1 — Inheritance

```
# Base class — shared behaviour
class BaseModule:
def __init__(self, name):
self.name = name
self.active = False

def start(self):
self.active = True
print(f'{self.name} started')

def stop(self):
self.active = False
print(f'{self.name} stopped')

def status(self):
state = 'ACTIVE' if self.active else 'INACTIVE'
print(f'{self.name}: {state}')

# Child class — inherits BaseModule and adds its own stuff
class MT5Connector(BaseModule):
def __init__(self):
super().__init__('MT5Connector') # run BaseModule.__init__
self.pairs = []

def load_pairs(self, pairs):
self.pairs = pairs
print(f'Loaded {len(pairs)} pairs')

conn = MT5Connector()
conn.status() # from BaseModule
conn.start() # from BaseModule
conn.load_pairs(['XAUUSD','GBPUSD','EURUSD','USDJPY'])
conn.status()
```

```
MT5Connector: INACTIVE
MT5Connector started
Loaded 4 pairs
MT5Connector: ACTIVE
```

Example 2 — How main.py Wires All Modules Together

```
# main.py — The conductor of the NAZON orchestra
from config import PAIRS, TIMEFRAME, DEFAULT_RISK

# (These would be real imports in full NAZON)
# from core.mt5_connector import MT5Connector
# from core.indicators import IndicatorEngine
# from core.risk_manager import RiskManager
# from ai.analyst import AIAnalyst
# from journal.logger import TradeLogger

def run_nazon():
print('===== NAZON STARTING =====')

# 1. Connect to MT5
# connector = MT5Connector()
# if not connector.connect(): return

# 2. Set up modules
# indicators = IndicatorEngine()
# risk = RiskManager(connector.balance, DEFAULT_RISK)
# analyst = AIAnalyst()
# logger = TradeLogger()

# 3. Scan all pairs
for pair in PAIRS:
print(f'Scanning {pair}...')
# df = connector.get_candles(pair, TIMEFRAME)
# ind = indicators.calculate(df)
# signal = analyst.analyse(pair, ind, connector.balance)
# lot = risk.calculate_lot(ind['atr'])
# logger.log(pair, signal, lot)

print('===== NAZON SCAN COMPLETE =====')

run_nazon()
```

```
===== NAZON STARTING =====
Scanning XAUUSD...
Scanning GBPUSD...
Scanning EURUSD...
Scanning USDJPY...
===== NAZON SCAN COMPLETE =====
```

LESSON 16

Decorators & Closures

NAZON uses decorators to add logging and retry logic without repeating code.

Example 1 — Basic Decorator

```
def log_it(func):
def wrapper(*args, **kwargs):
print(f'>>> Calling: {func.__name__}')
result = func(*args, **kwargs)
print(f'<<< Done: {func.__name__}')
return result
return wrapper

@log_it
def scan_pair(pair):
print(f' Scanning {pair}...')
return 'BUY'

signal = scan_pair('XAUUSD')
print(f'Signal: {signal}')
```

```
>>> Calling: scan_pair
Scanning XAUUSD...
<<< Done: scan_pair
Signal: BUY
```

Example 2 — Retry Decorator (NAZON MT5 Reconnect)

```
import time

def retry(attempts=3, delay=1):
def decorator(func):
def wrapper(*args, **kwargs):
for i in range(attempts):
try:
return func(*args, **kwargs)
except Exception as e:
print(f'Attempt {i+1} failed: {e}')
if i < attempts - 1:
time.sleep(delay)
print(f'{func.__name__} failed after {attempts} attempts')
return None
return wrapper
return decorator

attempt_num = 0

@retry(attempts=3, delay=0)
def connect_mt5():
global attempt_num
attempt_num += 1
if attempt_num < 3:
raise ConnectionError('MT5 terminal not responding')
print('Connected on attempt', attempt_num)
return True

connect_mt5()
```

```
Attempt 1 failed: MT5 terminal not responding
Attempt 2 failed: MT5 terminal not responding
Connected on attempt 3
```

LESSON 17

Threading & Scheduling

NAZON scans pairs automatically every 4 hours while the dashboard runs.

Example 1 — schedule Library

```
# pip install schedule
import schedule
import time

scan_count = 0

def scan_all_pairs():
global scan_count
scan_count += 1
print(f'Scan #{scan_count}: XAUUSD=BUY, GBPUSD=SELL, EURUSD=WAIT,
USDJPY=BUY')

# Run every 4 hours (in real NAZON)
# schedule.every(4).hours.do(scan_all_pairs)

# For testing — run every 2 seconds
schedule.every(2).seconds.do(scan_all_pairs)

# Run now, then wait
scan_all_pairs()

# Run 2 more times then stop
for _ in range(4):
schedule.run_pending()
time.sleep(1)
```

```
Scan #1: XAUUSD=BUY, GBPUSD=SELL, EURUSD=WAIT, USDJPY=BUY
Scan #2: XAUUSD=BUY, GBPUSD=SELL, EURUSD=WAIT, USDJPY=BUY
Scan #3: XAUUSD=BUY, GBPUSD=SELL, EURUSD=WAIT, USDJPY=BUY
```

Example 2 — Background Thread

```
import threading
import time

def background_scanner():
'''Runs in the background — doesn't block main app.'''
for i in range(3):
print(f'[SCANNER] Background scan #{i+1}')
time.sleep(1)
print('[SCANNER] Done.')

# Start the scanner in a background thread
thread = threading.Thread(target=background_scanner, daemon=True)
thread.start()

# Main thread continues running the dashboard
for i in range(3):
print(f'[DASHBOARD] Showing live data #{i+1}')
time.sleep(1)

thread.join()
print('Both finished!')
```

```
[DASHBOARD] Showing live data #1
[SCANNER] Background scan #1
[DASHBOARD] Showing live data #2
[SCANNER] Background scan #2
[DASHBOARD] Showing live data #3
[SCANNER] Background scan #3
[SCANNER] Done.
Both finished!
```

LESSON 18

SQLite Trade Journal

NAZON logs every trade to a database for AI to review your patterns.

Example 1 — Create Database and Table

```
import sqlite3

def init_database():
conn = sqlite3.connect('nazon_trades.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE IF NOT EXISTS trades (
id INTEGER PRIMARY KEY AUTOINCREMENT,
timestamp TEXT NOT NULL,
pair TEXT NOT NULL,
direction TEXT NOT NULL,
entry REAL,
stop_loss REAL,
take_profit REAL,
lot_size REAL,
outcome TEXT DEFAULT 'OPEN',
pnl REAL DEFAULT 0.0
)
''')
conn.commit()
conn.close()
print('Database ready!')

init_database()
```

```
Database ready!
```

Example 2 — Insert and Read Trades

```
from datetime import datetime

def log_trade(pair, direction, entry, sl, tp, lot):
conn = sqlite3.connect('nazon_trades.db')
cursor = conn.cursor()
cursor.execute(
'INSERT INTO trades
(timestamp,pair,direction,entry,stop_loss,take_profit,lot_size)'
' VALUES (?,?,?,?,?,?,?)',
(datetime.now().isoformat(), pair, direction, entry, sl, tp, lot)
)
conn.commit()
conn.close()
print(f'Logged: {pair} {direction}')

def get_trades():
conn = sqlite3.connect('nazon_trades.db')
cursor = conn.cursor()
cursor.execute('SELECT pair,direction,entry,outcome FROM trades')
rows = cursor.fetchall()
conn.close()
return rows

log_trade('XAUUSD','BUY', 2345.50,2321.00,2394.50,0.04)
log_trade('GBPUSD','SELL',1.2745, 1.2810, 1.2680, 0.10)

for row in get_trades():
print(row)
```

```
Logged: XAUUSD BUY
Logged: GBPUSD SELL
('XAUUSD', 'BUY', 2345.5, 'OPEN')
('GBPUSD', 'SELL', 1.2745, 'OPEN')
```

LESSON 19

MT5 Library Deep Dive

Complete reference for everything NAZON does with MetaTrader5.

Example 1 — Connect and Account Info

```
# pip install MetaTrader5 (MT5 desktop must be running)
import MetaTrader5 as mt5
import pandas as pd

# Connect
mt5.initialize()
mt5.login(123456789, 'YourPass', 'Exness-MT5Demo')

# Account info
info = mt5.account_info()
print(f'Balance : {info.balance}')
print(f'Equity : {info.equity}')
print(f'Profit : {info.profit}')
```

Example 2 — Fetch Candles and Live Price

```
# Fetch last 100 H4 candles for XAUUSD
rates = mt5.copy_rates_from_pos('XAUUSD', mt5.TIMEFRAME_H4, 0, 100)
df = pd.DataFrame(rates)
df['time'] = pd.to_datetime(df['time'], unit='s')
print(df.tail(3))

# Live bid/ask price
tick = mt5.symbol_info_tick('XAUUSD')
print(f'Bid: {tick.bid} Ask: {tick.ask}')
```

Example 3 — Open Positions and Place a Trade

```
# Check open positions
positions = mt5.positions_get()
for pos in positions:
print(f'{pos.symbol}: PnL={pos.profit}')

# Place a BUY trade
request = {
'action' : mt5.TRADE_ACTION_DEAL,
'symbol' : 'XAUUSD',
'volume' : 0.04,
'type' : mt5.ORDER_TYPE_BUY,
'price' : mt5.symbol_info_tick('XAUUSD').ask,
'sl' : 2321.00,
'tp' : 2394.50,
'comment' : 'NAZON AUTO',
'type_filling': mt5.ORDER_FILLING_IOC,
}
result = mt5.order_send(request)
print(f'Retcode: {result.retcode}')

mt5.shutdown()
```

■■ **WATCH OUT:** ALWAYS use your Exness Demo account when testing order_send(). Real money trades cannot be undone. Test for weeks on demo before going live.

Constant	Timeframe	NAZON Usage
mt5.TIMEFRAME_M1	1 minute	Very short scalp (not NAZON style)
mt5.TIMEFRAME_M15	15 minutes	Entry timing after signal
mt5.TIMEFRAME_H1	1 hour	Confirmation timeframe
mt5.TIMEFRAME_H4	4 hours	PRIMARY — swing trading
mt5.TIMEFRAME_D1	Daily	Trend direction confirmation

LESSON 20

NAZON v1 — Full Working Scanner

Everything from all 20 lessons combined into one working app.

Run this after connecting MT5 to your Exness demo account.

Part 1 — Imports, DB Setup & MT5 Connect

```
# nazon_v1.py — Full NAZON Scanner
# Combines: variables, f-strings, lists, dicts,
# conditionals, loops, functions, classes,
# error handling, pandas, .env, Claude API, MT5

import MetaTrader5 as mt5
import pandas as pd
import pandas_ta as ta
import anthropic
import sqlite3
from datetime import datetime
from dotenv import load_dotenv
import os

load_dotenv()

API_KEY = os.getenv('CLAUDE_API_KEY')
PAIRS = ['XAUUSD','GBPUSD','EURUSD','USDJPY']
TF = mt5.TIMEFRAME_H4

# ■■ 1. DATABASE SETUP ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def init_db():
conn = sqlite3.connect('nazon.db')
conn.cursor().execute('''
CREATE TABLE IF NOT EXISTS signals (
id INTEGER PRIMARY KEY AUTOINCREMENT,
time TEXT, pair TEXT, signal TEXT,
confidence INTEGER, rsi REAL, price REAL)
''')
conn.commit()
conn.close()

# ■■ 2. MT5 CONNECTION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def connect():
try:
if mt5.initialize():
print('MT5 Connected!')
return True
raise ConnectionError('initialize() failed')
except Exception as e:
print(f'MT5 Error: {e}')
return False
```

Part 2 — Indicators, AI Signal & Logging

```
# ■■ 3. FETCH CANDLES + INDICATORS ■■■■■■■■■■■■■■■■■■■■■■■■
def get_indicators(pair):
rates = mt5.copy_rates_from_pos(pair, TF, 0, 100)
df = pd.DataFrame(rates)
df['rsi'] = ta.rsi(df['close'], length=14)
df['ema200'] = ta.ema(df['close'], length=200)
macd = ta.macd(df['close'])
df['macd'] = macd['MACD_12_26_9']
row = df.iloc[-1]
trend = 'bullish' if row['close'] > row['ema200'] else 'bearish'
return {
'price' : round(row['close'],5),
'rsi' : round(row['rsi'], 2),
'macd' : round(row['macd'], 5),
'ema200': round(row['ema200'],2),
'trend' : trend,
}

# ■■ 4. CLAUDE AI SIGNAL ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def get_signal(pair, ind):
client = anthropic.Anthropic(api_key=API_KEY)
prompt = f'''
You are NAZON AI. Analyse {pair} for swing trading.
Price:{ind['price']} RSI:{ind['rsi']} MACD:{ind['macd']}
EMA200:{ind['ema200']} Trend:{ind['trend']}
Reply: SIGNAL:BUY/SELL/WAIT CONFIDENCE:0-100
'''
r = client.messages.create(
model='claude-sonnet-4-20250514',
max_tokens=60,
messages=[{'role':'user','content':prompt}]
)
return r.content[0].text.strip()
```

```
# ■■ 5. LOG TO DATABASE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def log_signal(pair, signal_text, ind):
conn = sqlite3.connect('nazon.db')
cursor = conn.cursor()
cursor.execute(
'INSERT INTO signals (time,pair,signal,rsi,price) VALUES (?,?,?,?,?)',
(datetime.now().isoformat(), pair, signal_text, ind['rsi'], ind['price'])
)
conn.commit()
conn.close()

# ■■ 6. RISK CHECK ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def calculate_lot(balance, risk_pct=0.01, sl_pts=200):
return round((balance * risk_pct) / (sl_pts * 0.1), 2)

# ■■ 7. MAIN SCANNER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def run():
print('=' * 40)
print(' NAZON FOREX ANALYSER — v1.0')
print('=' * 40)

init_db()
if not connect(): return

balance = mt5.account_info().balance
print(f'Account Balance: ${balance}')
```

Part 3 — Main Scanner Loop

```
print()

for pair in PAIRS:
try:
print(f'Scanning {pair}...')
ind = get_indicators(pair)
signal = get_signal(pair, ind)
lot = calculate_lot(balance)
log_signal(pair, signal, ind)

print(f' Price : {ind["price"]}')
print(f' RSI : {ind["rsi"]}')
print(f' Trend : {ind["trend"]}')
print(f' Signal : {signal}')
print(f' Lot : {lot}')
print()

except Exception as e:
print(f' Error on {pair}: {e}')

mt5.shutdown()
print('NAZON scan complete!')

if __name__ == '__main__':
run()
```

■ **NAZON CONNECTION:** This is NAZON v1. Every concept from all 20 lessons is here: imports, classes → functions, dicts, loops, try/except, pandas-ta, Claude API, SQLite and MT5. Phase 2 adds Streamlit dashboard. Phase 3 adds auto-execution.

QUICK REFERENCE — NAZON CHEATSHEET

Concept	Syntax / Example	Where in NAZON
Variable	price = 2345.50	Every file
f-string	f'RSI: {rsi}'	AI prompts, log messages
List	['XAUUSD','GBPUSD',...]	Pairs list, price history
Dict	{'rsi':68.5,'macd':0.001}	Signals, candles, account
if/elif/else	if rsi > 70:	signal_engine.py
for loop	for pair in pairs:	Scanning all 4 pairs
while loop	while running:	Main scan loop
function	def calculate_lot(...):	Every module
class	class MT5Connector:	Each module file
try/except	try: ... except:	MT5 calls, API calls
with open	with open('f.csv','a'):	Trade journal
pandas	df['close'].iloc[-1]	Candle data
pandas-ta	ta.rsi(df['close'],14)	indicators.py
.env / getenv	os.getenv('KEY')	config.py
Claude API	client.messages.create()	ai/analyst.py
SQLite	cursor.execute('INSERT...')	journal/logger.py
MT5 candles	mt5.copy_rates_from_pos()	mt5_connector.py
MT5 account	mt5.account_info().balance	risk_manager.py
MT5 trade	mt5.order_send(request)	executor.py (Phase 8)
Schedule	schedule.every(4).hours.do()	main.py loop

NAZON

You now have everything you need.

Practice every example. Break the code on purpose. Fix it. That's how real developers learn. When NAZON v1 runs and you see your first live XAUUSD signal come from Claude AI, you'll know it was worth it.

"The future depends on what you do today."

— Mahatma Gandhi

Now open VS Code and start typing. ■